

Laboratorio 3

FreeRTOS

Angelo Gauthier

Pilar Martínez

Ignacio Negri

Objetivo

Se utilizará FreeRTOS (sistema operativo en tiempo real) para el desarrollo de una arquitectura de software que implemente multitasking, partiendo de los requerimientos del Laboratorio 2. Se aplicarán diferentes mecanismos de FreeRTOS: tasks, queues, semaphores, timers de software y memoria dinámica.

Introducción

Se escribirá un programa que conste de tres tareas: task A, task B y task C. La primera de ellas constará de una prioridad baja y su fin será hacer parpadear el led con el color actual. Habrá un semáforo de acceso al color y el led iniciado en `app_main()` pasará el handle resultante a task A como parámetro de creación.

La task B tendrá prioridad alta, recibirá comandos por UART y los encolará. Se programará un protocolo de comandos, parseará la línea completa, construirá un `led_command_t` y lo enviará a la cola. Enviará un mensaje de confirmación tras encolar exitosamente.

Finalmente, la task C será de prioridad media y se encargará de desencolar comandos y crear timers one-shot. Creará un timer one-shot por cada comando que se ingrese. Asignará dinámicamente un bloque de memoria y copiará el color en él. Se utilizará un parámetro `pvTimerID` como puntero al bloque de memoria. Hará un callback para recuperar el puntero al color, tomar el mutex, escribir el nuevo color en la variable compartida, liberar el mutex, liberar la memoria y eliminar el timer desde el callback mismo.

Creación de Tareas

La creación de tareas se realizó siguiendo el ejemplo de la página de ESP-IDF, en la cual se utiliza la función de FreeRTOS `xTaskCreate` (*FreeRTOS (IDF) - ESP32 - — ESP-IDF Programming Guide v6.0.1 Documentation*, s. f.) que lleva como parámetros los descritos en la Figura 1.

<code>psTaskCode</code>	Puntero a la función principal de la tarea
<code>pcName</code>	Nombre descriptivo de la tarea
<code>usStackDepth</code>	Tamaño del stack asignado a la tarea, en bytes
<code>pvParameters</code>	Puntero a los parámetros que se le pasarán a la tarea al momento de ser creada
<code>ucPriority</code>	Prioridad de ejecución de la tarea.
<code>pxCreatedTask</code>	Parámetro utilizado para devolver el handle de la tarea creada

Figura 1: parámetros utilizados en la creación de las tres tareas principales

Semaforos

Un semáforo es un mecanismo que permite acceder de forma segura a un recurso compartido. Cuidando que dos tareas no puedan acceder a una variable y modificarla al mismo tiempo. En el caso de este laboratorio se utilizaron semáforos binarios (mutex), que es un semáforo contador inicializado en 1 y garantiza que solo una tarea pueda acceder al recurso compartido a la vez (*Modulo 2 Introduccion y Conceptos Basicos*, 2026).

Esto se utiliza con las siguientes funciones:

- `xSemaphoreTake()`: una tarea toma el mutex antes de acceder al recurso (ninguna tarea puede usarlo una vez tomado)
- `xSemaphoreGive()`: se libera después de utilizar el recurso.

En específico en este laboratorio, se utilizó un mutex para proteger la variable global que almacena el color actual del LED (`current_color`).

Task A

Es la tarea encargada de controlar el parpadeo del led rgb en la ESP32-S2. Lee periódicamente el color actual almacenado en una variable compartida y lo utiliza para prender y apagar el led de manera intermitente. Para poder acceder al color actual, la tarea A recibe como parámetro una estructura `task_a_params_t` definida en `task_a.h`. Esta contiene un puntero a `current_color` y un `color_mutex` para proteger el acceso concurrente a la variable compartida.

Dentro de la función de la tarea, el parámetro recibido como `pvParameters` se convierte a un puntero de tipo `task_a_params_t`. Primero, se verifica que los parámetros sean válidos (que ni el puntero al color ni el mutex sean NULL). Si esto ocurre, la tarea muestra una etiqueta de tipo `ESP_LOGE`, que indica un error que impide el correcto funcionamiento, y se elimina a sí misma mediante `vTaskDelete(NULL)`.

Se utiliza un `while(1)` para obtener un bucle infinito. Dentro del bucle, si solo la tarea A quiere obtener acceso al mutex (por ser de prioridad baja), lo toma con `xSemaphoreTake`, copia el valor de `current_color` a una variable local `color_local` y luego libera el mutex con `xSemaphoreGive`. Si no puede acceder al mutex muestra una etiqueta de tipo `ESP_LOGW` que indica situaciones inesperadas no fatales. La copia en la variable local permitirá que la tarea A use el color sin mantener bloqueado el recurso compartido.

Una vez leído el color, llama a `rgb_led_set_color(color_local.r, color_local.g, color_local.b)` para prender el led con los valores correspondientes de `r`, `g` y `b` actuales. Luego, tiene un delay programado con `vTaskDelay(pdMS_TO_TICKS(500))` de 500 ms, apaga el led con `rgb_led_off()` y nuevamente tiene un delay del mismo tiempo que el anterior antes de repetir el proceso.

Finalmente, en `main.c` se inicializa el led con `rgb_led_init`, se crea el mutex con `xSemaphoreCreateMutex()` y se utiliza la estructura `task_a_params_t` como `task_a_params` y se le asocia con la dirección de `g_current_color` y `g_color_mutex`. Después, se crea la tarea A mediante `xTaskCreate`, asignándole un stack de 4096 bytes y la prioridad más baja (`tskIDLE_PRIORITY + 1`).

Task B

Es la tarea encargada de recibir comandos por UART ingresados desde el monitor, interpretarlos y enviarlos mediante una cola de FreeRTOS. Funciona como la entrada del sistema, toma los comandos, los transforma en una estructura `led_command_t` y los pone en la cola.

En el programa, recibe como parámetro un `g_command_queue` de tipo `QueueHandle_t` creado previamente en `main.c`. Allí mismo se crea la cola utilizando:

```
xQueueCreate(10, sizeof(led_command_t))
```

Esto permite almacenar hasta 10 comandos pendientes. A continuación, se crea la tarea B con `xTaskCreate` y se le asigna una prioridad alta de `tskIDLE_PRIORITY + 3`. Se realiza un chequeo para comprobar que la tarea fue correctamente creada, mostrando en la consola una etiqueta de tipo `ESP_LOGE` si no lo fue y una

etiqueta de tipo `ESP_LOGI`, indicando un evento normal, si fue correctamente creada. Además, en caso de no crearse, sale de `app_main()`.

Dentro del código de la tarea B (archivo: `task_b.c`), se configura la UART definiendo `UART_PORT` como `UART_NUM_0` al igual que el laboratorio anterior. Se configura una velocidad de comunicación de 115200 bits por segundo, 8 bits para representar cada carácter y un bit de stop para indicar el final de cada dato. Para inicializarla se utilizan estas tres funciones:

- 1) **uart_driver_install**: instala el driver, “asigna los recursos internos necesarios para el controlador UART” (*Universal Asynchronous Receiver/Transmitter (UART) - ESP32-S2 - — ESP-IDF Programming Guide v6.0.1 Documentation*, s. f.)
- 2) **uart_param_config**: se configuran los parámetros mostrados en la Figura 2
- 3) **uart_set_pin**: configura los pines GPIO físicos a los que se conectara la ESP32-S2

Parámetro a configurar	Función
Velocidad en baudios	<code>uart_set_baudrate()</code>
Número de bits transmitidos	<code>uart_set_word_length()</code> seleccionado de entre <code>uart_word_length_t</code>
Control de paridad	<code>uart_set_parity()</code> seleccionado de entre <code>uart_parity_t</code>
Número de bits de parada	<code>uart_set_stop_bits()</code> seleccionado de entre <code>uart_stop_bits_t</code>
Modo de control de flujo de hardware	<code>uart_set_hw_flow_ctrl()</code> seleccionado de entre <code>uart_hw_flowcontrol_t</code>
Modo de comunicación	<code>uart_set_mode()</code> seleccionado de entre <code>uart_mode_t</code>

Figura 2: parámetros configurados utilizando la función (2) descrita anteriormente

Luego se pasa a un bucle el cual describe el protocolo utilizado para la interacción con el usuario a través del monitor. El protocolo consta de escribir un color seguido de una cantidad de segundos. Se leen los caracteres uno por uno usando `uart_read_bytes`, los va guardando en un buffer y marca como finalizado el comando cuando recibe un espacio (`\n`). Una vez que se recibe una línea, la tarea B intenta interpretarla con:

```
sscanf(line, "%19s %u", color_str, &delay_s)
```

Esa línea se encarga de separar el comando recibido en dos partes, el color (se almacena como un string) y el tiempo de delay (se almacena como un entero). Luego se utiliza una función estática, `parse_color`, la cual compara el string recibido con una lista de colores acotada (ROJO, AZUL, VERDE y BLANCO). Si coincide llena la struct `rgb_color_t` (definida en `shared_types.h`) con los valores RGB. Si no coincide utiliza un log (`ESP_LOGW`) indicando que el color no fue reconocido.

Una vez que se confirma que el comando es válido, se completa la otra estructura `cmd` (de tipo `led_command_t` definida en `shared_types.h`) que consta del comando que luego es enviado a la cola utilizando:

```
xQueueSend(queue, &cmd, portMAX_DELAY)
```

Finalmente se muestra en el monitor a través de un LOG que el comando fue recibido y enviado a la cola.

Task C

Es la encargada de administrar los cambios de color diferidos del LED RGB utilizando timers de FreeRTOS. Su función principal es recibir comandos desde una cola, crear timers asociados a cada color y actualizar el color global cuando el tiempo configurado expira.

El flujo principal de la task_c se puede describir de la siguiente manera:

QUEUE → xQueueReceive() → identificar color → verificar timer existente → eliminar timer anterior (si existe) → crear color_copy → xTimerCreate() → xTimerStart() → esperar expiración → timer_callback() → tomar mutex → actualizar color global → liberar recursos

Al inicio se incluyen las librerías necesarias para trabajar con timers y logs del sistema. Luego se define un arreglo llamado `color_timers`, que almacena los timers activos de cada color. Cada posición del arreglo representa un color específico, permitiendo saber rápidamente si ya existe un timer activo para ese color y reemplazarlo si es necesario (opcional cancelación de timers pendientes).

Para identificar cada color se utiliza un enum llamado `color_id_t`, donde se asigna un identificador numerico a ROJO, VERDE, AZUL y BLANCO. Esto simplifica el acceso al arreglo de timers mencionado anteriormente.

La función `get_color_id()` recibe un color RGB y devuelve el identificador correspondiente. De esta manera, el sistema puede relacionar un valor RGB con la posición correcta dentro del arreglo de timers. También se implementa la función `color_to_string()`, utilizada únicamente para mostrar mensajes más claros en los logs del sistema.

La tarea utiliza dos variables globales estáticas: `g_current_color` apunta al color actual compartido del sistema, mientras que `g_color_mutex` protege el acceso concurrente a dicho color. Estas variables son necesarias porque el callback del timer también debe acceder al color global.

Sobre el funcionamiento del callback, la función `timer_callback()` es ejecutada automáticamente por el servicio interno de timers de FreeRTOS cuando un timer expira. El callback obtiene el color asociado al timer utilizando: `rgb_color_t *new_color = (rgb_color_t *)pvTimerGetTimerID(xTimer);`.

Cada timer almacena un puntero a un color específico, permitiendo saber qué color aplicar cuando el tiempo finaliza. Luego el callback toma el mutex para acceder de forma segura al color compartido, esto evita condiciones de carrera entre TASK A y el callback.

El mutex asegura acceso exclusivo al recurso compartido. Sin mutex podrían ocurrir conflictos, donde una tarea lee el color mientras otra lo modifica simultáneamente. Por eso las tasks a y c, que son las que necesitan el recurso del color actual, utilizan mutex para protegerlo momentáneamente.

Una vez tomado el mutex, el callback actualiza el color global. Y lo libera utilizando `xSemaphoreGive(g_color_mutex);`

Cuando el timer termina, también es necesario liberar los recursos asociados. Primero se elimina la referencia del timer en el arreglo con `color_timers[id] = NULL`. Después se libera la memoria dinámica reservada para el color, y finalmente el timer es destruido; ambas utilizando funciones específicas del RTOS: `vPortFree(new_color);` y `xTimerDelete(xTimer, 0);` respectivamente.

Pasando a la función principal de la tarea, `void task_c(void *pvParameters)`, que al iniciar, obtiene la cola de comandos, el color global y el mutex, con la forma `task_c_params_t`.

Luego entra en un loop infinito esperando comandos. La tarea permanece bloqueada hasta recibir un comando válido desde la cola. Esto evita polling constante y reduce el uso innecesario de CPU.

Cuando se recibe un comando, lo primero que se hace es verificar si ya existe un timer activo para ese color en `color_timers[id]`. Si existe, el timer anterior es eliminado y reemplazado por el nuevo. El flujo de trabajo para la asignación dinámica de timers se puede diagramar de la siguiente manera (figura 3):

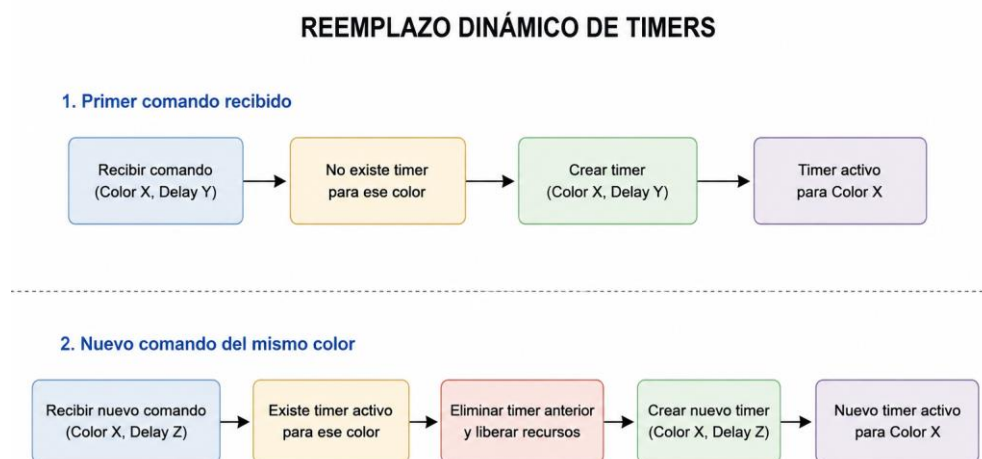


Figura 3: Diagrama de flujo de reemplazo dinámico de timers en TASK C.

Además, antes de eliminar el timer anterior se libera la memoria dinámica asociada al color viejo. Esto permite actualizar delays sin acumular timers innecesarios. En la queue no se reemplaza nada ya que en esta solo se guardan comandos de transmisión de B a C.

Siguiendo con la creación del timer, antes de crearlo se reserva memoria dinámica para almacenar una copia persistente del color. Esto es necesario porque el timer puede ejecutarse mucho después de que la variable local original deje de existir. Luego se copia el color con `*color_copy = cmd.color;`

Luego de esto, para la creación del timer se utiliza:

```
color_timers[id] = xTimerCreate, "ColorTimer", pdMS_TO_TICKS(cmd.delay_s * 1000), pdFALSE, color_copy, timer_callback);
```

Características importantes:

- **pdFALSE** indica que el timer es one-shot.
- **pdMS_TO_TICKS()** convierte segundos a ticks de FreeRTOS.
- **color_copy** se guarda como identificador del timer.
- **timer_callback** es la función ejecutada al expirar.

Luego el timer se inicia con `xTimerStart(color_timers[id], 0);`, a partir de ese momento FreeRTOS comienza a contar el tiempo internamente.

Como conclusión, podemos describir el flujo completo de la TASK C de la siguiente manera:

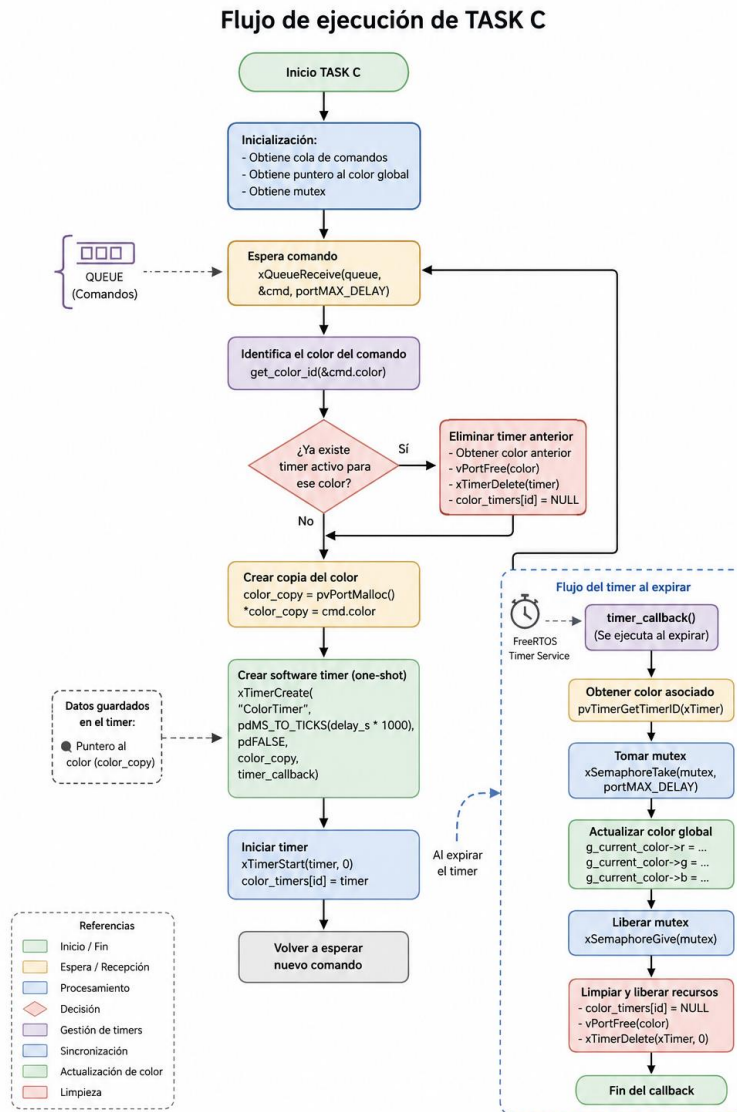


Figura 4: Flujo de trabajo de TASK C

Propuestas de extensión:

- Cancelación de Timers pendientes:

Como opcional, se agregó un sistema para evitar que existan varios timers activos del mismo color al mismo tiempo. La idea es, si llega un nuevo comando para un color que ya tiene un timer esperando, el timer viejo se elimina y se reemplaza por el nuevo.

Por ejemplo:

Llega ROJO 10 → timer rojo creado

Llega ROJO 3 → timer anterior eliminado → nuevo timer rojo creado

Para hacer esto se usa un arreglo llamado `color_timers`, donde se guarda el timer activo de cada color. Cuando TASK C recibe un comando:

1. identifica el color,

2. verifica si ya existe un timer para ese color,
3. si existe, lo elimina y libera su memoria,
4. y luego crea el nuevo timer actualizado.

Este funcionamiento evita timers duplicados, uso innecesario de memoria y cambios de color viejos que ya no interesan.

Además, se agregaron mensajes con `ESP_LOGI()` para mostrar cuándo un timer es reemplazado, facilitando la depuración del sistema.

El flujo de trabajo de esta asignación de timers de forma dinámica, se puede entender perfectamente con el diagrama de la figura 3.

- Monitor de Stack:

La idea fue monitorear el stack de cada tarea utilizando la función de FreeRTOS `uxTaskGetStackWaterMark()` y ajustar en la creación de cada tarea el `usStackDepth` al mínimo necesario. La función anterior sirve para definir cuanto espacio libre le queda al stack de una tarea.

Para esto, se agregó un monitoreo en los archivos `task_b.c` y `task_c.c`, luego de procesar comandos válidos y luego de crear timers respectivamente. De esta manera, en el monitor aparece la cantidad de stack libre en bytes debido a:

```
uxTaskGetStackHighWaterMark(NULL) * sizeof(StackType_t)
```

A partir de los valores obtenidos, se observó que las tareas mantenían una cantidad de espacio libre (ej. I (36305) TASK_C: Stack libre: 2472 bytes), por lo que se redujo en el `main.c` (utilizando `#define`) el parámetro `usStackDepth` en cada `TaskCreate()`, evitando utilizar memoria innecesaria.

Inicialmente, el monitoreo también se aplicó en la tarea A. Sin embargo, como es una tarea que se ejecuta de forma periódica y continua, los logs de `task_a` aparecían mucho en el monitor e interferían con el envío de los comandos.

Por ese motivo, se utilizó únicamente el monitoreo en las últimas dos tareas modificando los valores de `usStackDepth` que inicialmente eran 4096 a:

- `usStackDepth_A : 2048`
- `usStackDepth_B : 3072`
- `usStackDepth_C : 3072`

Conclusiones

Para concluir, se logro el objetivo del laboratorio utilizando conceptos clave de FreeRTOS como las tareas, las colas, los semáforos, los timers de software y la memoria dinámica. La implementación de tareas independientes con distintas prioridades permitió mejorar la organización dentro del grupo a la hora de realizar el laboratorio. Además, a partir de eso se logró llegar a los resultados esperados.

De acuerdo con las prioridades de cada tarea, la tarea B al ser la que se encarga de la interacción con el usuario, tiene sentido que tenga la mayor prioridad. Se logro implementar la comunicación serial y funciona de manera que es capaz de recibir, interpretar y enviar comandos como se esperaba en la letra. La tarea C se encarga de crear y manejar los timers one-shot, a la vez que también administra los

cambios de color (con delay) utilizando memoria dinámica. Tanto en esta tarea como la A se implementó el uso semáforos para utilizar la variable compartida. Finalmnete, la tarea A, la de menor prioridad se encarga del parpadeo periódico del LED utilizando dicha variable mencionada anteriormente.

A su vez, también se lograron realizar dos de las propuestas de extensión. Estas mejoran el comportamiento global del sistema. Por un lado, se agregó el reemplazo dinámico de timers, el cual modifica el timer de un color en la fila si este se encuentra duplicado. Por otro lado, se cambió la memoria de stack de cada tarea correspondiendo a el espacio que deja libre la misma. De esta manera, con ambas adiciones, se logra optimizar el uso de la memoria.

Finalmente, el laboratorio sirvió para entender de forma práctica cómo funciona FreeRTOS trabajado no solo con varias tareas al mismo tiempo, sino que con tareas en simultaneo de distinta prioridad utilizando colas, mutex y timers. También, permitió ver cómo organizar mejor un sistema embebido para que cada tarea tenga una función específica (prender led, recibir comandos, manejar tiempos) de manera de generar un programa que funcione de forma más ordenada y eficiente.

Bibliografía:

- *Modulo 2 Introduccion y conceptos basicos*. (2026, mayo). [Diapositivas 35-37].
webasignatura.ucu.edu.uy.
https://webasignatura.ucu.edu.uy/pluginfile.php/1164065/mod_resource/content/14/M%C3%B3dulo%202%20-%20SisEm.pdf
- *FreeRTOS (IDF) - ESP32 - — ESP-IDF Programming Guide v6.0.1 documentation*. (s. f.).
https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos_idf.html#_CPPv427uxTaskGetStackHighWaterMark12TaskHandle_t
- *Universal Asynchronous Receiver/Transmitter (UART) - ESP32-S2 - — ESP-IDF Programming Guide v6.0.1 documentation*. (s. f.). <https://docs.espressif.com/projects/esp-idf/en/stable/esp32s2/api-reference/peripherals/uart.html>
- *uxTaskGetStackHighWaterMark, UXTaskGetStackHighWaterMark2 - FreeRTOS™*. (s. f.). <https://www.freertos.org/Documentation/02-Kernel/04-API-references/03-Task-utilities/04-uxTaskGetStackHighWaterMark>
-